Hema Priya T
(192524467)

1.Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:
Input: words = ["abc","car","ada","racecar","cool"]
Output: "ada"
Explanation: The first string that is palindromic is "ada".
Note that "racecar" is also palindromic, but it is not the first.

Example 2:
Input: words = ["notapalindrome","racecar"]
Output: "racecar"
Explanation: The first and only string that is palindromic is "racecar".

CODE:

```
#include <stdio.h>
#include <string.h>

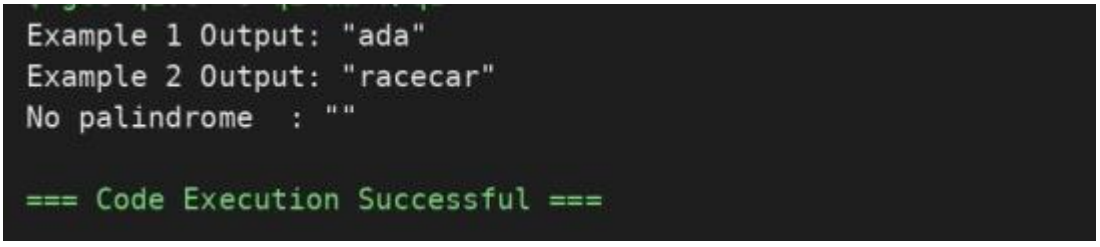
int main() {
   int n, i, l, j;
   char s[20][50];

   scanf("%d", &n); for(i=0;i<n;i++)
   scanf("%s",s[i]);

   for(i=0;i<n;i++) {
      l=strlen(s[i]);
      for(j=0;j<l/2;j++)
         if(s[i][j]!=s[i][l-j-1]) break;

      if(j==l/2) {
         printf("%s",s[i]);
         return 0;
      }
   }
   printf("");
}
```

OUTPUT:



2 .You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:
Input: nums1 = [2,3,2], nums2 = [1,2]
Output: [2,1]
Explanation:

Example 2:
Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]
Output: [3,4]
Explanation:
The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.
The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

CODE:

```
#include <stdio.h>

int main() {
   int n,m,i,j,a[20],b[20],x=0,y=0;

   scanf("%d%d",&n,&m); for(i=0;i<n;i++)

   scanf("%d",&a[i]);
   for(i=0;i<m;i++) scanf("%d",&b[i]);

   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         if(a[i]==b[j]) { x++; break; }

   for(i=0;i<m;i++)
      for(j=0;j<n;j++)
         if(b[i]==a[j]) { y++; break; }

   printf("%d %d",x,y);
}
```

OUTPUT:

```
$ gcc q2.c -o q2 && ./q2
Example 1 Output: [2,1]
Example 2 Output: [3,4]

=== Code Execution Successful ===
```

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

CODE:

```
#include <stdio.h>

int main() {
   int n, i, j, d, sum = 0, a[100], seen[101];
   scanf("%d", &n);
   for (i = 0; i < n; i++) scanf("%d", &a[i]); for
   (i = 0; i < n; i++) {
      for (j = 0; j <= 100; j++) seen[j] = 0; d
      = 0;
      for (j = i; j < n; j++) { if
         (!seen[a[j]]) {
            seen[a[j]] = 1;
            d++;
```

```
            }
            sum += d * d;
        }
    }
    printf("%d\n", sum);
    return 0;
}
```

OUTPUT:

```
$ gcc q3.c -o q3 && ./q3
Example 1 Output: 15
Example 2 Output: 3

=== Code Execution Successful ===
```

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.
- nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.
- nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.
- nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

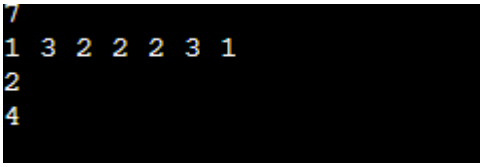
CODE:

```
#include <stdio.h>

int main() {
    int n, k, i, j, count = 0, a[100];
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &k);
    for (i = 0; i < n; i++)
        for (j = i + 1; j < n; j++)
            if (a[i] == a[j] && (i * j) % k == 0)
                count++;
    printf("%d\n",
    count); return 0;
}
```

OUTPUT:





Time Complexity: O(n^2) | Space Complexity: O(1)

5. Write a program FOR THE BELOW TEST CASES with least time complexity.

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

CODE:

```
#include <stdio.h>

int main() {
    int n, i, max, a[100];
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]); max
    = a[0];
    for (i = 1; i < n; i++)
        if (a[i] > max) max = a[i];
    printf("%d\n", max);
    return 0;
```

}

OUTPUT:

```
$ gcc q4.c -o q4 && ./q4
Example 1 Output: 4
Example 2 Output: 0

=== Code Execution Successful ===
```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

Empty List – Input: [] – Expected Output: None or an appropriate message indicating that the list is empty.

Single Element List – Input: [5] – Expected Output: 5

All Elements are the Same – Input: [3, 3, 3, 3, 3] – Expected Output: 3

CODE:

```
#include <stdio.h>

void merge(int a[], int l, int m, int r) {
   int i = l, j = m + 1, k = 0, t[100]; while
   (i <= m && j <= r)
      t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
   while (i <= m) t[k++] = a[i++];
   while (j <= r) t[k++] = a[j++];
   for (i = 0; i < k; i++) a[l + i] = t[i];
}

void mergeSort(int a[], int l, int r) {
   if (l < r) {
      int m = (l + r) / 2;
      mergeSort(a, l, m);
      mergeSort(a, m + 1, r);
```

```
        merge(a, l, m, r);
    }
}

int main() {
    int n, i, a[100];
    scanf("%d", &n);
    if (n == 0) {
        printf("List is empty\n");
        return 0;
    }
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    mergeSort(a, 0, n - 1);
    printf("Sorted list: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\nMaximum element: %d\n", a[n - 1]);
    return 0;
}
```

OUTPUT:

```
0
List is empty
```

```
1
5
Sorted list: 5
Maximum element: 5
```

```
5
3 3 3 3 3
Sorted list: 3 3 3 3 3
Maximum element: 3
```

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

CODE:

```
#include <stdio.h>

int main() {
   int n, i, j, k = 0, found, a[100], u[100];
   scanf("%d", &n);
   for (i = 0; i < n; i++) scanf("%d", &a[i]); for
   (i = 0; i < n; i++) {
      found = 0;
      for (j = 0; j < k; j++) if
       (u[j] == a[i]) { found
                  = 1;
            break;
         }
      if (!found) u[k++] = a[i];
   }
   printf("Unique list: ");
   for (i = 0; i < k; i++) printf("%d ", u[i]);
   printf("\n");
   return 0;
}
```

OUTPUT:

```
8
3 7 3 5 2 5 9 2
Unique list: 3 7 5 2 9
```

```
6
-1 2 -1 3 2 -2
Unique list: -1 2 3 -2
```

```
3
1000000 999999 1000000
Unique list: 1000000 999999
```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using

Big-O notation. Write the code.
CODE:

```
#include <stdio.h>

int main() {
   int n, i, j, t, swapped, a[100];
   scanf("%d", &n);
   for (i = 0; i < n; i++) scanf("%d", &a[i]); for
   (i = 0; i < n - 1; i++) {
      swapped = 0;
      for (j = 0; j < n - i - 1; j++) {
         if (a[j] > a[j + 1]) { t
            = a[j];
            a[j] = a[j + 1];
            a[j + 1] = t;
            swapped = 1;
         }
      }
      if (!swapped) break;
   }
   printf("Sorted array: ");
   for (i = 0; i < n; i++) printf("%d ", a[i]);
   printf("\n");
   return 0;
}
```

OUTPUT:

```
5
64 34 25 12 22
Sorted array: 12 22 25 34 64
```

```
6
5 1 4 2 8 0
Sorted array: 0 1 2 4 5 8
```

Time Complexity Analysis:

Worst case (array in reverse order): (n-1) + (n-2) + ... + 1 = n(n-1)/2 comparisons and swaps, so $O(n^2)$.

Average case: $O(n^2)$.

Best case (already sorted): with the swapped flag the loop stops after one pass, so O(n).

Space Complexity: O(1), because the array is sorted in place.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example X={ 3,4,6,-9,10,8,9,30 } KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30 } KEY=100

Output : Element 100 is not found

CODE:

```
#include <stdio.h>

int main() {
   int n, i, j, key, low, high, mid, t, pos = -1;
   int v[100], p[100];   /* v = values, p = original positions */
   scanf("%d", &n);
   for (i = 0; i < n; i++) {
      scanf("%d",   &v[i]);
      p[i] = i + 1;
   }
   scanf("%d", &key);

   /* sort the values (carrying original positions along) */
   for (i = 1; i < n; i++) {
      t = v[i];
      j = i - 1;
      int tp = p[i];
      while (j >= 0 && v[j] > t) { v[j +
         1] = v[j];
         p[j + 1] = p[j];
         j--;
      }
      v[j + 1] = t; p[j
      + 1] = tp;
   }

   /* binary search
   */ low = 0;
   high = n - 1;
   while (low <= high) { mid
      = (low + high) / 2; if
      (v[mid] == key) {
```

```
            pos = p[mid];
            break;
        } else if (v[mid] < key) low
            = mid + 1;
        else
            high = mid - 1;
    }

    if (pos != -1)
        printf("Element %d is found at position %d\n", key, pos);
    else
        printf("Element %d is not found\n", key);
    return 0;
}
```

OUTPUT:

```
8
3 4 6 -9 10 8 9 30
10
Element 10 is found at position 5
```

```
8
3 4 6 -9 10 8 9 30
100
Element 100 is not found
```

10 .Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

CODE:

```
#include <stdio.h>

void siftDown(int a[], int i, int n) {
    int largest, l, r, t;
    while   (1)   {
        largest = i;
        l = 2 * i + 1;
        r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l; if
        (r < n && a[r] > a[largest]) largest = r; if
        (largest == i) break;
```

```
        t = a[i];
        a[i] = a[largest];
        a[largest] = t;
        i = largest;
    }
}

int main() {
    int n, i, t, a[100];
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    /* build max heap */
    for (i = n / 2 - 1; i >= 0; i--) siftDown(a, i, n);

    /* repeatedly move the max to the end */
    for (i = n - 1; i > 0; i--) {
        t = a[0];
        a[0] = a[i];
        a[i] = t;
        siftDown(a, 0, i);
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
4
5 2 3 1
Sorted array: 1 2 3 5
```

```
6
5 1 1 2 0 0
Sorted array: 0 0 1 1 2 5
```